

Engram 교체 실험(2)

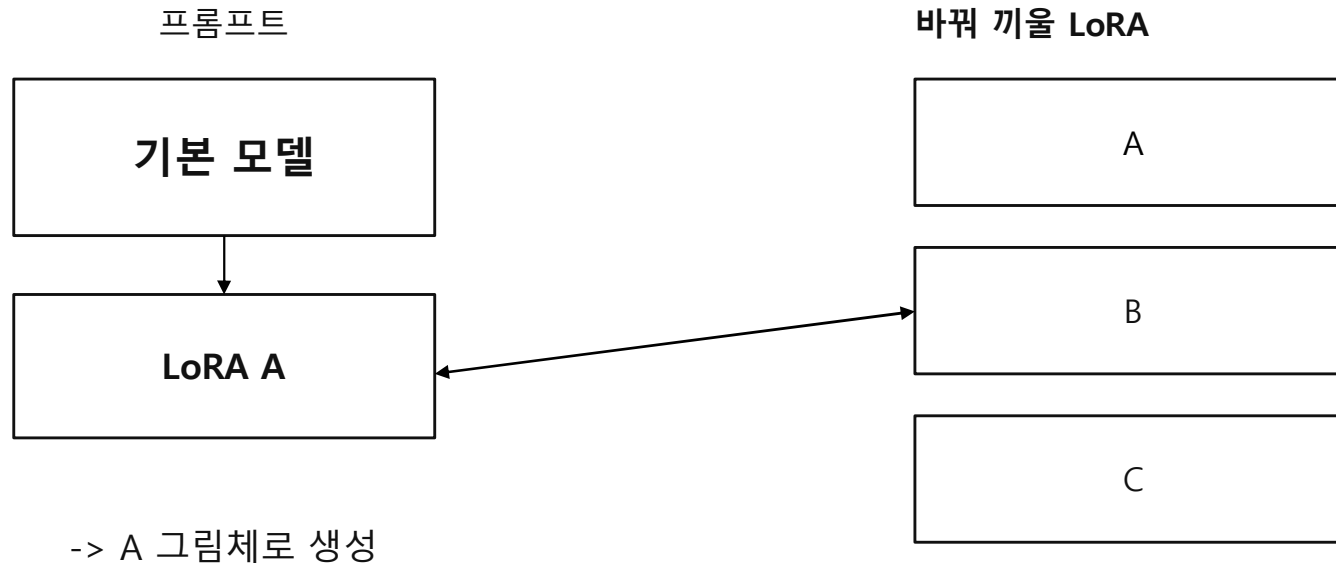
20220300 김종현

2026-10-08

목차

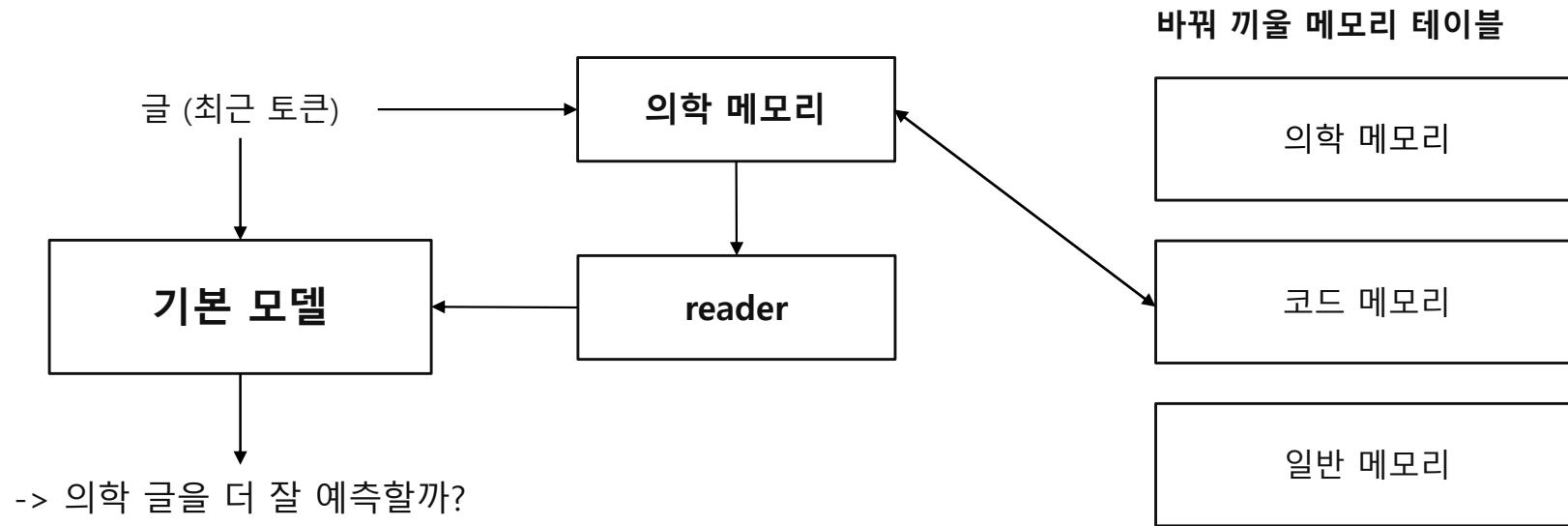
1. 왜 Engram 메모리 교체인가
2. 이전 실험: 새 메모리는 어떻게 학습해야 효율적인가
3. 이번 실험 설계: 모델 · 데이터 · 평가
4. 결과: 메모리를 바꿔 끼우면?
5. 정리

이미지 생성 모델에는 LoRA가 자주 사용된다



- 이미지 생성 모델은 특이하게도 커뮤니티 위주로 발달
- VRAM의 크기보다 자체 성능이 더 중요 -> 접근성도 좋고 여러 사람들에 의해 사용법이 연구됨
- 모델은 놔두고 LoRA만 여러 개를 학습하여 다양하게 만드는 방향으로 발달

Engram 메모리도 같이 끼울 수 있을까?



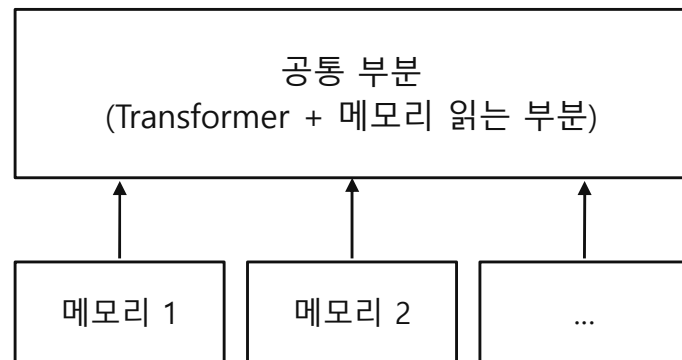
- Engram: 최근 토큰 조합으로 큰 메모리 테이블을 찾아 모델에 더하는 구조
- 메모리 테이블이 본체와 따로 있음 -> LoRA처럼 분야별 테이블만 바꿔 끼우면?

이전 실험: 새 메모리는 어떻게 학습해야 할까?

교체하려면 새 메모리를 계속 만들어야 함 -> 공통 부분은 어떻게 준비해 두는 게 좋을까?

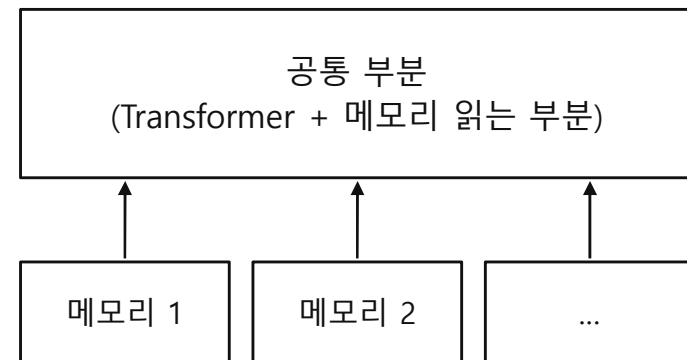
Baseline

공통 부분 고정 · 메모리만 학습



Joint

여러 메모리를 번갈아 끼우며 공통 부분도 함께 학습



- 질문: Joint로 준비한 공통 부분이 새 메모리를 더 빨리 받아들일까?
- 비교: Engram을 쓰도록 미리 학습한 같은 작은 모델에서 준비 방식만 다르게 -> 새 메모리 학습 속도

가상 사실 실험에서는 공통 부분을 고정해도 충분했다

규모 · 비율	Joint – Baseline (학습 속도 차이)	Joint가 앞선 반복
작은 규모 · 비율 1	-0.111	0 / 9
작은 규모 · 비율 10	-0.054	0 / 9
중간 규모 · 비율 1	-0.046	0 / 9
중간 규모 · 비율 10	+0.009	7 / 9
큰 규모 · 비율 1	-0.012	1 / 9
큰 규모 · 비율 10	+0.002	6 / 9

- Baseline도 새 사실을 거의 100% 회상
- Joint는 작은 규모에서 오히려 느렸고, 앞선 경우도 차이가 아주 작음
- Joint의 추가 준비 비용(약 42초~11분)을 새 메모리 몇 개로는 회수 못함

그래서 이번 실험은

이전 실험을 보고 택한 설계

공통 부분은 한 번 준비해 고정
새 메모리는 테이블만 학습

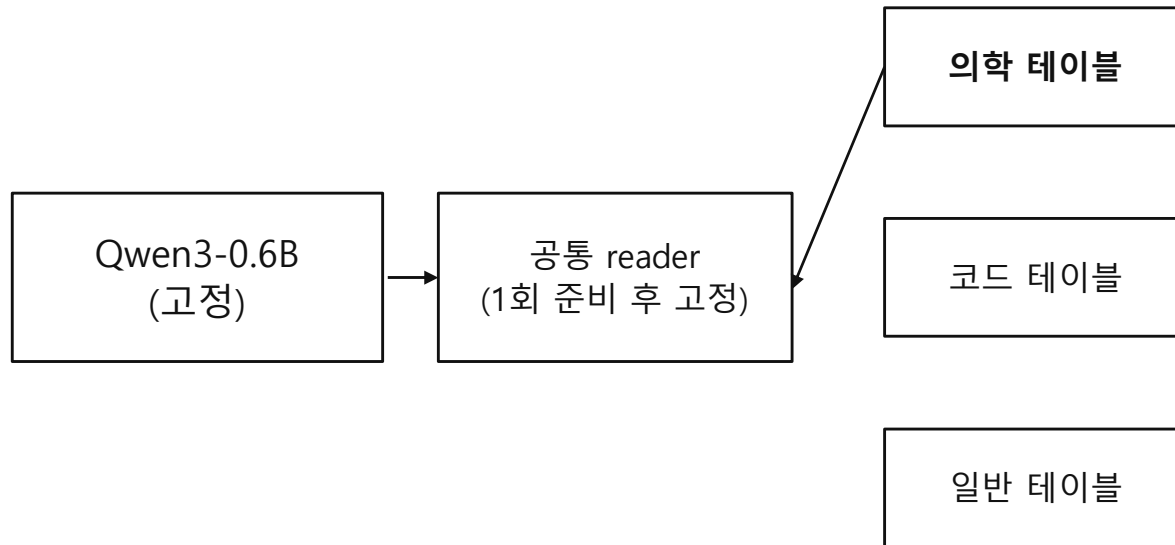


이번에 확인할 것

진짜 언어모델(Qwen3-0.6B)과
진짜 분야 글(의학 · 코드)에서도
테이블만 바꿔 끼워 특화되나?

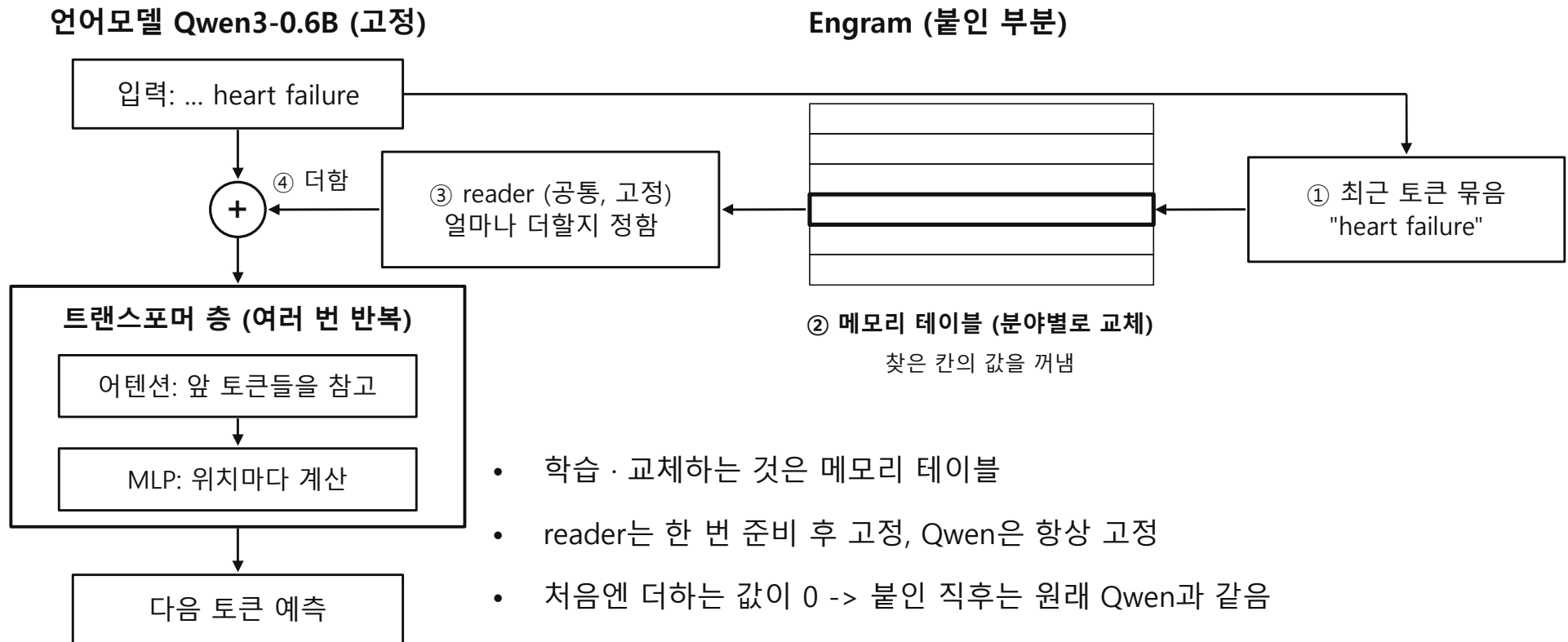
- 이전 실험: 뜻 없는 가상 사실 · 처음부터 학습한 작은 모델
- 이번 실험: 사전학습된 언어모델에 Engram을 붙이고, 분야별 메모리만 교체

무엇을 확인하고 싶은가?

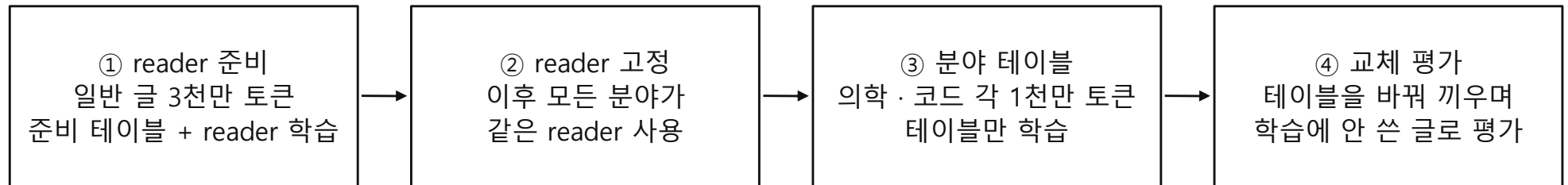


- 목표: 메모리만 갈아끼우면 그 분야에 특화 (이미지 모델의 LoRA처럼)
- 이번 질문: 공통 부분은 그대로, 분야 테이블만 학습해 끼우면 그 분야 글을 더 잘 예측하나?
- LoRA보다 좋을 필요는 없음 -> 작은 환경에서 효과가 있는지 · 얼마나 · 비용은?

Engram을 어떻게 붙였나?



무엇을 어떤 순서로 학습했나?



단계	Qwen3	reader	메모리 테이블
① 준비	고정	학습	준비 테이블 학습
③ 분야 적응	고정	고정	분야 테이블만 학습

주 실험: 분야 테이블을 0에서 시작
-> 끼운 직후 출력 = 원래 Qwen

기존 연구와 무엇이 다른가?

	무엇을 바꿔 끼우나	무엇을 학습 · 고정하나
Conditional Memory Adapter (Hou & Wang, 2026)	분야 테이블 + 읽는 부분	분야마다 함께 학습
이번 실험	분야 테이블 전체	reader는 일반 글로 1회 준비 후 고정 분야마다 테이블만 학습

어떤 데이터를 썼나?

용도	데이터	학습 (사용 / 확보)	개발용	평가용	나눔 단위
reader 준비 · 일반 평가	FineWeb-Edu (교육용 웹 글)	3천만 / 6천만	100만	200만	문서
의학	PubMed 논문 제목 + 초록	1천만 / 4천만	100만	200만	논문 (PMID)
코드	CodeSearchNet Python 함수	1천만 / 4천만	100만	177만	저장소

- 같은 논문 · 같은 저장소의 글이 학습과 평가에 동시에 들어가지 않게 나눔
- Qwen이 사전학습 때 본 글과 겹치는지는 확인하지 못함
- 모두 공개 데이터 · 재배포 없이 연구용 평가에만 사용

데이터는 실제로 어떻게 생겼나?

의학: PubMed 제목 + 초록

Characteristics of Lower Leg Muscle Activity in Patients with Cerebral Palsy during Cycling on an Ergometer.
Cycling on ergometer is often part of rehabilitation programs for patients with cerebral palsy (CP). The present study analyzed activity patterns of individual lower leg muscle during active cycling on ergometer in patients with CP and compared them to similar recordings in healthy participants. Electromyographic (EMG) recordings ...

평가용 5,824편 · 길이 중앙값 약 330토큰

코드: CodeSearchNet Python

```
def _chosen_css(self):
    """Read the minified CSS file
    including STATIC_URL in the
    references to the sprite
    images."""
    css = render_to_string(
        self.css_template, {})
    for sprite in self.chosen_sprites:
        css = css.replace(sprite,
            settings.STATIC_URL
            + "img/" + sprite)
    return css
```

평가용 8,066개 함수 · 길이 중앙값 약 150토큰

일반: FineWeb-Edu 웹 글

One of the most active volcanic regions in the world has seen another eruption. The eruption of Puyehue-Cordón Caulle, Chile started on the 4th June and has led to the evacuation of ~3500 from its vicinity.
The eruption was preceded by an increase in fumarolic activity and a swarm of earthquakes on the 27th April which were sourced at ~4-6 km below the surface. Closer to the 4th the earthquakes began to increase in intensity ...

평가용 1,703편 · 길이 중앙값 약 740토큰

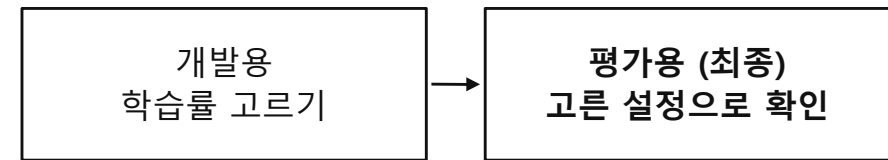
- 모두 영어 · 글 한 편(함수 하나)이 학습 · 평가의 한 단위, 같은 논문 · 저장소는 학습과 평가에 나눠 넣지 않음
- 일반 글에도 건강 · 의학 관련 글이 섞여 있음 -> 일반 글로 만든 준비 테이블도 의학 글을 꽤 개선한 이유 중 하나일 수 있음

어떻게 평가했나?

예측 손실 (bpb): 글 1바이트를 맞히는 데 드는 비트 수 · 정답을 높은 확률로 예상할수록 작아짐
낮을수록 그 글을 더 잘 예측

판정 = 같은 분야 글 안에서 비교

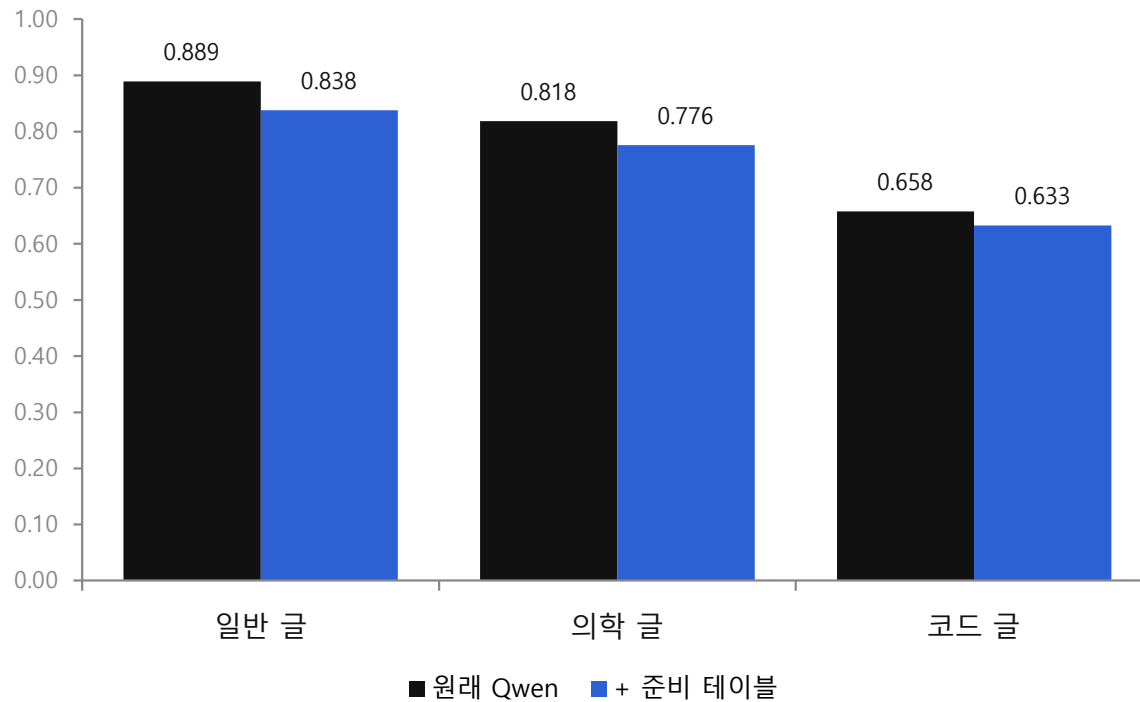
의학 글에서	비교 대상
의학 테이블과	원래 Qwen (테이블 없음)
의학 테이블과	코드 테이블 (잘못 끼움)
의학 테이블과	준비 테이블 (일반 글로만 학습) · 추가 확인



- 주 실험: 평가 결과를 본 뒤 설정을 다시 고르지 않음
- 테이블 학습은 데이터 추출 · 순서를 바꿔 3번 반복

먼저 일반 준비 테이블만 끼워도?

예측 손실 (낮을수록 좋음)



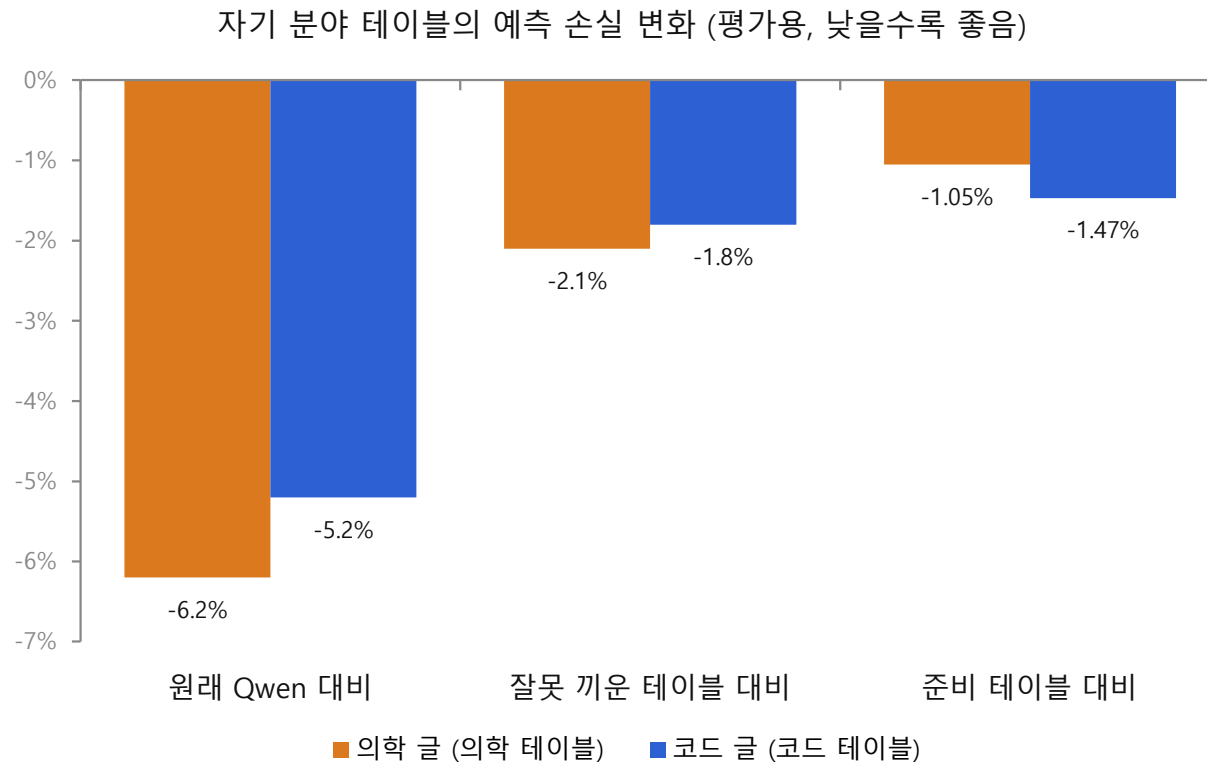
- 일반 글로만 학습한 테이블도 세 분야 모두 개선
- 일반 -5.8% · 의학 -5.2% · 코드 -3.8%
- -> 원래 Qwen 대비 개선 전부를 분야 특화로 볼 수는 없음
- 그래서 분야 테이블은 준비 테이블보다도 나아야 의미

메모리를 바꿔 끼우면?

끼운 테이블 ↓ 평가 글 ->	일반 글	의학 글	코드 글
없음 (원래 Qwen)	0.8894	0.8183	0.6577
준비 테이블	0.8380	0.7758	0.6326
의학 테이블	0.8436	0.7676	0.6348
코드 테이블	0.8486	0.7845	0.6232

- 의학 글에서는 의학 테이블, 코드 글에서는 코드 테이블이 가장 좋음
- 테이블만 바꿨는데 그 분야로 특화 되는 대각선 모양
- 일반 글은 준비 테이블이 가장 좋음 (분야 테이블은 약간 손해)

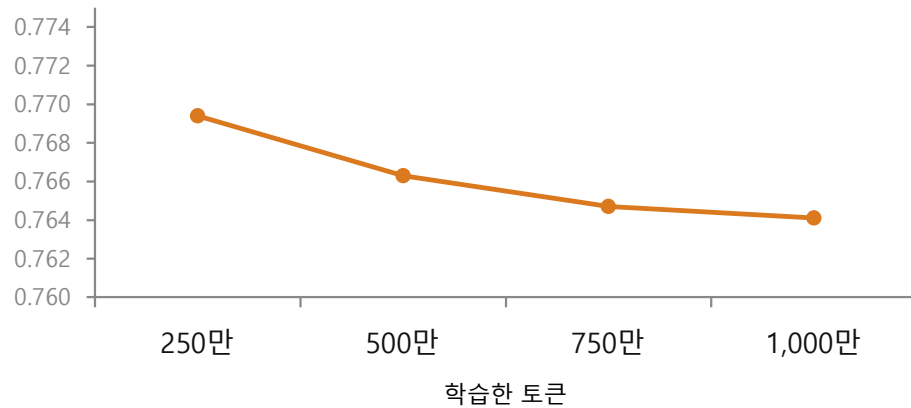
분야 안에서 비교하면?



- 3번 반복 모두 같은 방향
- 통계 구간(95%)으로 봐도 손실 차이가 모두 0 미만
- 잘못 끼운 테이블보다 약 2% 좋음 = 분야에 맞는 테이블을 고른 이점
- 준비 테이블보다도 약 1~1.5% 좋음

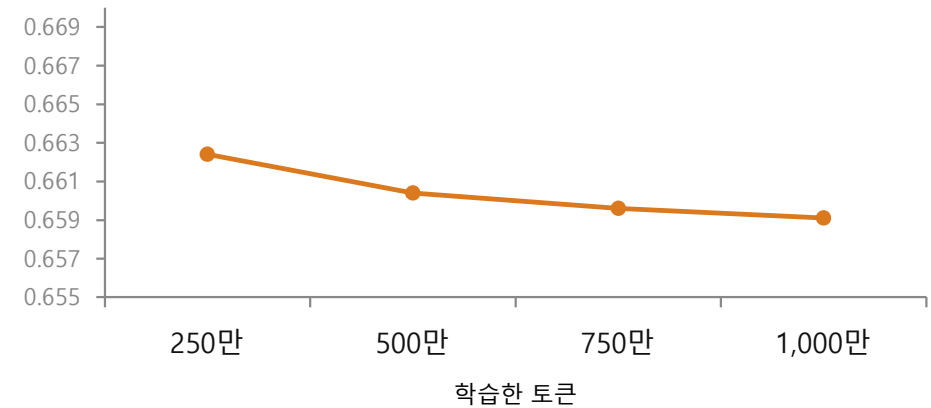
더 많이 학습하면 좋아질까?

의학 개발용 예측 손실



— 분야 테이블 (0에서 시작, 1회)

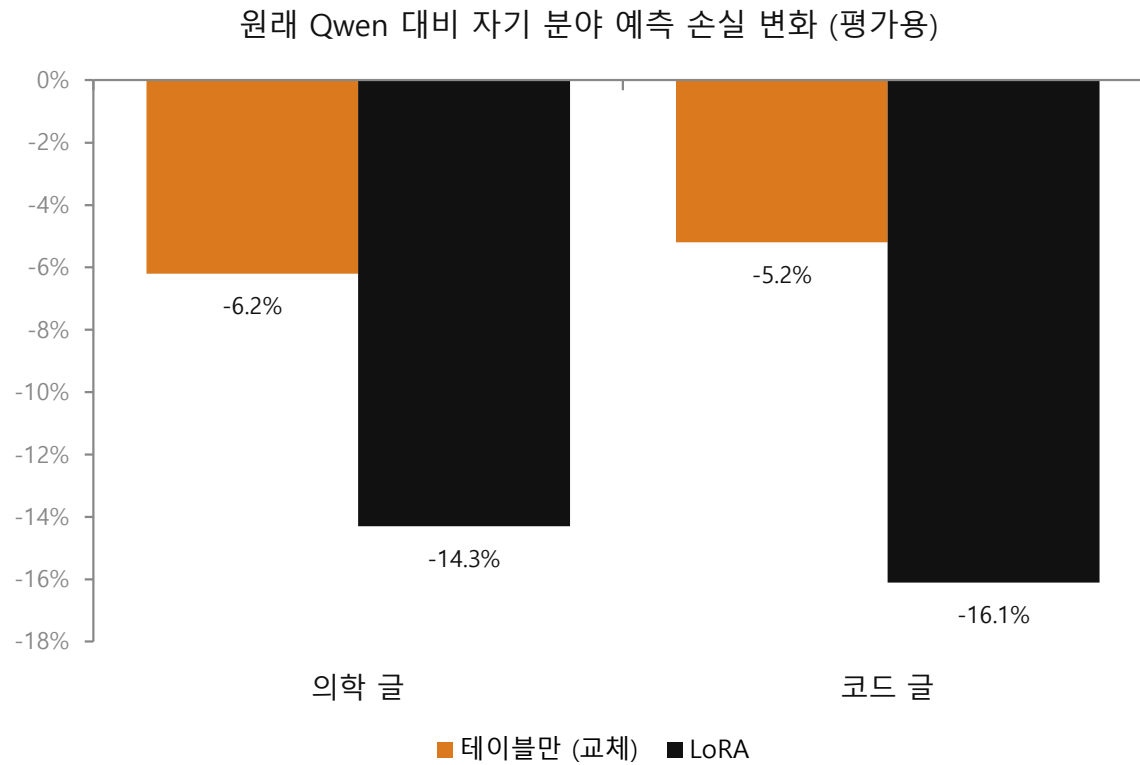
코드 개발용 예측 손실



— 분야 테이블 (0에서 시작, 1회)

- 1,000만 토큰까지 계속 줄지만 후반은 거의 평평
- 250만 토큰에서 이미 전체 개선의 약 90%
- 개발용 · 1회 학습 결과, 학습량을 더 늘린 실험은 아직 안 함

다른 학습 방법과 비교하면?



- 같은 원래 모델 · 같은 분야 데이터 · 같은 1천만 토큰
- LoRA는 개선 폭이 2~3배 (파라미터 수는 1.2배)
- 성능적으로 본다면 Engram보다는 Lora가 더 우위

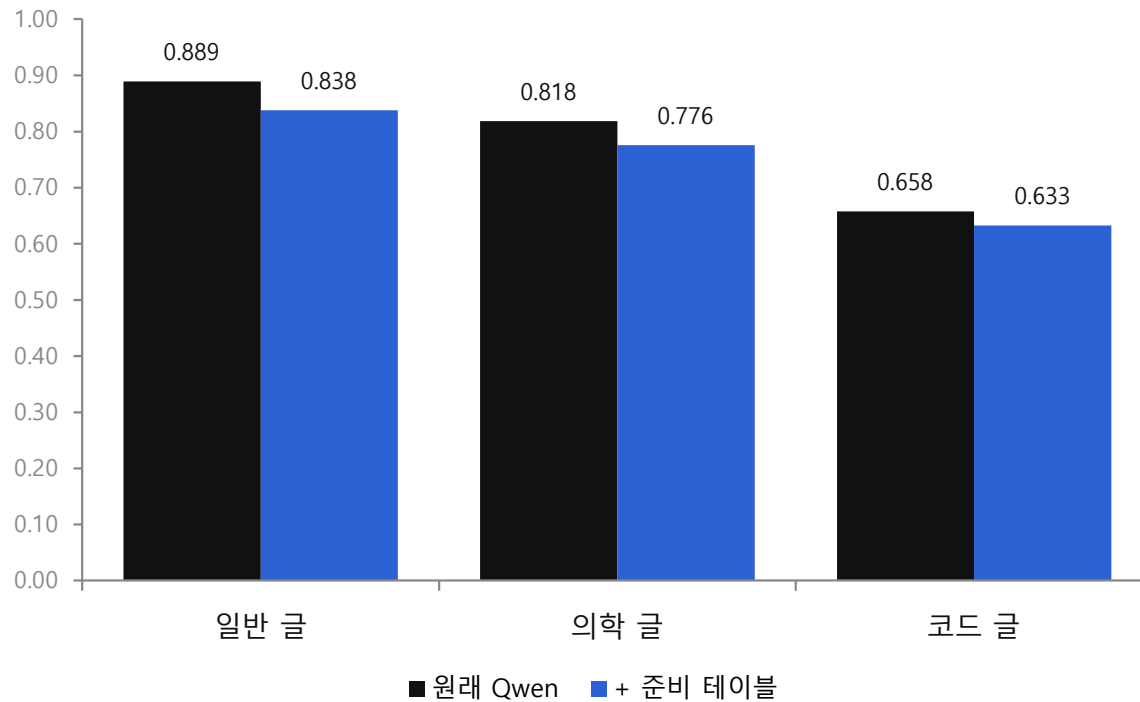
비용은 어떤가?

	GPU 시간	학습 파라미터	저장 크기	GPU 메모리 (최대 할당)
공통 reader 준비 (1번만)	46분	3,619만	145MB	7.9GiB
분야 추가: 테이블만	약 16분	3,356만	134MB	7.8GiB
분야 추가: LoRA	약 22분	4,037만	162MB	10.3GiB

- 분야 하나 추가만 보면 테이블이 LoRA보다 시간 -28% · 저장 -17% · 메모리 -24%
- 하지만 reader 준비 46분이 먼저 필요 -> 2분야 합계는 Engram 77분, LoRA 44분
- 단순 계산상 8분야부터 준비 비용 회수

결론

예측 손실 (낮을수록 좋음)



- 현재 Engram 방식으로 학습한 메모리 테이블을 바꾸는 것으로 효과가 있음을 확인
- LoRA와 비교에서 성능적으로 낮은 모습을 보여주지만 비용적인 측면에서 개선
- 아직 추가로 실험하고 싶은 부분 (준비 테이블을 복사하여 학습, 테이블 크기 조절, 학습량 등)
- 최종적으로는 테이블 교체 구조